



Full Stack Development Week 06

Topics covered:

- Asynchronous operations (calling an API using Fetch)
- useRef
- Custom hook

Activity 1: Asynchronous operation (call an API with fetch)

As discussed during lectional 5, you will often have to call an external API to fetch data. We discussed the example of Google's API during the lectional. Let us do something different now:

A community-maintained list of public / free API's can be found here:

<https://github.com/public-apis/public-apis>

We will use one of the API used can be found here:

<https://random.dog/>

Making a call to the Url: <https://random.dog/woof.json> returns a different dog each time.

Making use of the following:

- `useEffect` hook
- `fetch()`

Make an asynchronous call to the API and display the result on the web page.

Explore both of the following ways to call the API asynchronously:

- `then/catch`
- `async/await`

Which of these approaches is better and why?

Activity 2: useRef in action

Let's create a fun "Whack-a-Mole" style game using `useRef`! In this game, you'll need to click on randomly appearing targets as quickly as possible.

1. Create a new component called `WhackAMole.tsx` with the following features:
 - A game board with 9 squares (3x3 grid)

- A score counter
- A timer that counts down from 30 seconds
- A "Start Game" button

2. Using `useRef`, implement the following functionality:

- Track the game timer using `useRef` to prevent the timer from resetting on re-renders
- Use `useRef` to access DOM elements for the mole squares
- Track the current score using `useRef` to ensure it persists between renders

3. Game mechanics:

- When the game starts, moles (or any fun emoji of your choice) appear randomly in squares
- Clicking a mole increases your score
- Moles disappear after 1 second if not clicked
- The game ends after 30 seconds

This activity will help you understand:

- How `useRef` can persist values between renders
 - How to use `useRef` to access DOM elements
 - How to manage cleanup of intervals and event listeners
 - The difference between `useRef` and `useState`
-

Activity 3: Custom hook

Building upon Activity 1, let's create a reusable custom hook for fetching data from APIs. This will help us understand how to abstract common functionality into custom hooks.

1. Create a new file called `useFetch.ts` and implement a custom hook called `useFetch` that:

- Takes a URL as a parameter
- Returns an object containing:
 - `data`: The fetched data
 - `loading`: A boolean indicating if the fetch is in progress
 - `error`: Any error that occurred during fetching
 - `refetch`: A function to manually trigger a refetch

2. Use this custom hook in a new component called `DogFetcher.tsx` to:

- Display a random dog image using the same API from Activity 1
- Show a loading state while fetching
- Display any errors that occur
- Include a "Get New Dog" button that uses the `refetch` function

3. Compare this implementation with Activity 1:

- What are the benefits of using a custom hook?
- How does it improve code reusability?
- What other use cases can you think of for this custom hook?

Activity 4: React Query Hook

In this activity, we'll explore TanStack Query (formerly React Query), a powerful library for managing async state in React applications. We'll build upon our previous activities to see how TanStack Query can simplify data fetching and caching.

1. First, install TanStack Query in your project:

```
npm install @tanstack/react-query
```

2. Create a new component called `DogQuery.tsx` that uses TanStack Query to:

- Fetch random dog images using the same API from previous activities
- Display the image with loading and error states
- Include a "Get New Dog" button that triggers a refetch
- Show how many times the data has been fetched

3. Implement the following features:

- Use `useQuery` hook to fetch the data
- Configure the query with:
 - A unique query key
 - A custom fetch function
- Use the built-in loading and error states

4. Compare this implementation with Activities 1 and 3:

- What are the advantages of using TanStack Query over manual fetching?
- How does it handle caching and refetching automatically?
- What other features does TanStack Query provide that would be useful in a real application?